Unidad 10 Ficheros de Texto

Flujos de entrada de texto
Scanner
Flujos de salida de texto

Los flujos de entrada de tipo texto se heredan de la clase **InputStreamReader**. Todas las clases que vamos a usar para trabajar con ficheros se encuentran en el paquete **java.io**.

Podemos importarlas todas con la sentencia,

```
import java.io.*;
```

Las clases de entrada de texto tienen siempre un nombre que termina en Reader. Nosotros usaremos flujos del tipo **FileReader**. El constructor es:

```
FileReader(String nombreArchivo)
```

al que se le pasa, como parámetro, el nombre del archivo al que se quiere asociar un flujo para su lectura. Este nombre puede llevar incluida la ruta de acceso si el archivo no está en la carpeta de trabajo. Por ejemplo,

```
FileReader in = new FileReader("C:\\programas\\prueba.txt");
```

crea un flujo de texto asociado al archivo **prueba.txt**, que se halla en la carpeta programas de la unidad C. No hay que olvidar que, para imprimir la barra invertida, hay que escribirla doble\\), ya que escrita de forma simple se emplea para las secuencias de escape(por ejemplo, «\n» significa un cambio de línea). Cuando estamos trabajando en una plataforma Linux, las rutas de acceso son distintas.

Por ejemplo

```
FileReader in = new FileReader("/home/pedro/programas/prueba.txt");
//la barra hacia delante (/) se puede escribir simple
```

La apertura de un fichero puede arrojar una excepción del tipo **IOException** (Input-Output Exception, excepción de entrada y salida) (o alguna subclase, cuando el archivo no se abre por alguna razón.

Por ejemplo, el constructor de **FileReader** puede arrojar una excepción **FileNotFoundException**, que hereda de **IOException** si el archivo que queremos abrir para lectura no existe, o no se encuentra en la ruta de acceso especificada. Por ello, dicha operación siempre deberá ir dentro de la estructura try-catch correspondiente. Cuando se abre un archivo, un cursor se posiciona al principio, apuntando al primer carácter, e irá avanzando por el archivo a medida que vayamos leyendo de él.

Una vez que se ha abierto el fichero para lectura, podremos leer del flujo asociado, usando el siguiente método:

int read(): lee del fichero y devuelve un carácter Unicode codificado como un entero.

Si queremos recuperar el carácter que lleva dentro cada entero, debemos aplicarle un cast, ya que la conversión, al ser de estrechamiento, no es automática.

Sabremos que hemos llegado al final del archivo cuando **read() devuelva -1**, que no corresponde a ningún caracter.

void close(): cierra el flujo de entrada con objeto de completar las lecturas pendientes y liberar el archivo.

Ejemplo

```
FileReader in = null;
try {
        in = new FileReader("Main.java");
        ...
} catch(IOException ex) {
        System.out.println(ex.getMessage());
}
```

FileReader nos permite leer cualquier archivo de texto plano creado con un editor de texto como NotePad o Gedit (no con un procesador de texto, como Word).

Sin embargo, por razones de eficiencia, no se suele usar **FileReader** tal cual.

Normalmente se emplean flujos de la clase **BufferedReader**, que no es más que un FileReader filtrado para asociar un búfer (espacio reservado para almacenamiento temporal) en memoria. Esto permite hacer lecturas en el dispositivo físico (el disco, por ejemplo) de grupos de caracteres, en vez de caracteres individuales. Estos son colocados en cola en el búfer, a la espera de que el programa los vaya reclamando. Con ello se reduce el número de accesos al

disco, que es una operación extremadamente lenta. Para crear un BufferedReader, basta pasarle al constructor un objeto FileReader.

```
BufferedReader in = new BufferedReader(new FileReader("Main.java"));
```

Ahora, el flujo in dispone del método **read()** para hacer las lecturas de caracteres individuales, pero, además, al tener un búfer asociado, puede hacer lecturas de líneas completas con el siguiente método:

String readLine(): lee y devuelve líneas completas del archivo de texto, sin incluir el cambio de línea del final. Al llegar al final del fichero devuelve **null**.

Scanner

Hasta ahora hemos usado la clase Scanner para leer datos introducidos por el teclado, pero este, en realidad, es un flujo de entrada de texto (System. in). Lo que hace la clase Scanner es acceder al búfer del teclado en busca de secuencias de caracteres (llamadas tokens) que se adapten al tipo de datos requeridos por el método invocado: next(), nextInt (), etc.
Esto supone que Scanner, además de leer secuencias de caracteres, es capaz de analizarlas y convertirlas en datos de tipos primitivos. Por defecto, para que un objeto Scanner identifique los distintos tokens, estos deben estar separados por caracteres blancos (espacios, tabuladores o cambios de línea) o secuencias de ellos.

Ejemplo

Si queremos leer del teclado una serie de cinco números enteros, y guardarlos en una tabla, podemos escribir:

```
int[] tabla=new int[5];
Scanner s = new Scanner(System.in);
System.out.print("Introducir serie de 5 enteros: ");
for (int i = 0; i < 5; i++) {
        int n = s.nextInt();
        tabla[i]=n;
}
System.out.println(Arrays.toString(tabla));
```

Al ejecutar el programa, es posible introducir los números de uno en uno, pulsando Intro cada vez, o podemos escribirlos todos separados por espacios en una sola línea, por ejemplo: 1 34 22 o 143 y pulsar Intro después.

En ambos casos obtendremos por pantalla la serie de los números introducidos

```
[1, 34, 22, 0, 143]
```

Si hemos introducido todos los números en una misma línea, en cada llamada a **nextInt()**, el Scanner localiza los espacios separadores y reconoce cada token (grupo de caracteres no blancos), lo analiza, identifica un entero, lo convierte en un dato de tipo int y lo devuelve. La sentencia de asignación correspondiente lo asigna a la variable n.

NOTA: Si queremos usar el punto decimal en vez de la coma al leer un número real con Scanner, debemos establecer como localización un país que la use, como Estados Unidos.

```
Scanner s = new Scanner(System.in).useLocale(Locale.US);
```

Igual que hemos usado Scanner para leer y analizar una secuencia de caracteres tecleados en una línea, podemos analizar una cadena de caracteres. En este caso no leeremos del flujo procedente del búfer del teclado (**System.in**), sino de un objeto String, que le pasaremos al constructor de Scanner.

```
String numeros = "1 34 22 0 143";
Scanner s = new Scanner(numeros);
```

El resto del programa sería igual y produciría los mismos resultados.

Ahora podemos usar la clase Scanner para analizar el contenido de un archivo de texto. Lo abrimos creando un flujo de tipo **BufferedReader** y analizamos las cadenas devueltas por el método **readLine()**, línea a línea.

Cabe preguntarse si, dado que el teclado es un flujo de caracteres, Scanner puede acceder a otros flujos de texto, como los que estamos estudiando en esta unidad. La respuesta es sí: basta con pasar al constructor de Scanner el flujo asociado a un archivo de texto, con su ruta de acceso si procede.

La clase Scanner es útil cuando un archivo de texto contiene tokens que representan datos numéricos que hay que identificar y codificar para luego hacer cálculos.

Flujos de salida de texto

Si en vez de leer de un archivo de texto queremos escribir en él, necesitaremos un flujo de salida de texto. Para ello, crearemos un objeto de la clase **FileWriter**, que hereda de **OutputStreamWriter-**

Los constructores de FileWriter son:

```
FileWriter(String nombreArchivo )
FileWriter(String nombreArchivo , boolean append)
```

donde **nombreArchivo**, como ya ocurría en **FileReader**, puede contener la ruta de acceso. El primer constructor destruye la versión anterior del archivo y escribe en él desde el principio.

Sin embargo, el booleano **append**, cuando vale true, nos permite añadir texto al final del archivo, respetando el contenido anterior. La apertura de un **FileWriter** puede generar una excepción del tipo **IOException**, que habrá que tratar con el try-catch correspondiente.

Como hicimos con **FileReader**, para mejorar el rendimiento usaremos una versión con búfer, **BufferedWriter**, que guarda los caracteres en un búfer. Cuando este está lleno, se graban en la unidad de almacenamiento correspondiente. Al constructor de **BufferedWriter** se le pasa como parámetro un flujo de salida de tipo **FileWriter**.

```
BufferedWriter out;
out = new BufferedWriter(new FileWriter("salida.txt"));
```

Los métodos de que disponemos con **BufferedWriter** son:

void write(int caracter): escribe un carácter en el archivo.
void write(String cadena): escribe una cadena en el archivo.
void newLine(): escribe un salto de línea en el fichero. Se debe evitar el uso explícito del carácter «\n» para insertar saltos de líneas, ya que su codificación es distinta según la plataforma utilizada.
void flush(): vacía el búfer de salida, escribiendo en el fichero los caracteres pendientes.
void close(): cierra el flujo de salida, vaciando el buffer y liberando el recurso correspondiente.

Es común olvidarse de cerrar el flujo. Como resultado podemos encontrarnos un archivo incompleto o incluso vacío. Esto es porque los caracteres se han guardado en el búfer, pero no han llegado a escribirse en el archivo antes de que el

programa termine. Por eso e s importante no olvidar el bloque finally con el cierre del flujo.

Sin embargo, **a partir de Java 7 disponemos de una estructura para cerrar archivos o liberar cualquier recurso, sin necesidad de usar el método close() ni el bloque finally**. Se trata de la estructura try-catch-resources o apertura con recursos

Ejemplo

```
try(BufferedWriter out = new BufferedWriter(new FileWriter("quijote.txt"))) {
        String cad = "En un lugar de La Mancha,"; //primera línea
        for (int i = 0; i < cad.length(); i++) {
                out.write(cad.charAt(i));
        }
        cad = "de cuyo nombre no quiero acordarme."; //segunda línea
        out.newLine(); //cambio de línea en el archivo
        out.write(cad);
} catch (IOException ex) {
        System.out.println(ex.getMessage());
}
```

Tanto si se produce la excepción como si no, el flujo in se cierra automáticamente al terminar de ejecutarse la estructura try-catch. En el paréntesis que sigue a try, se pueden abrir varios flujos. Basta separar las sentencias de apertura con un signo “;”.